

Plan Assurance Qualité – Projet ESP800

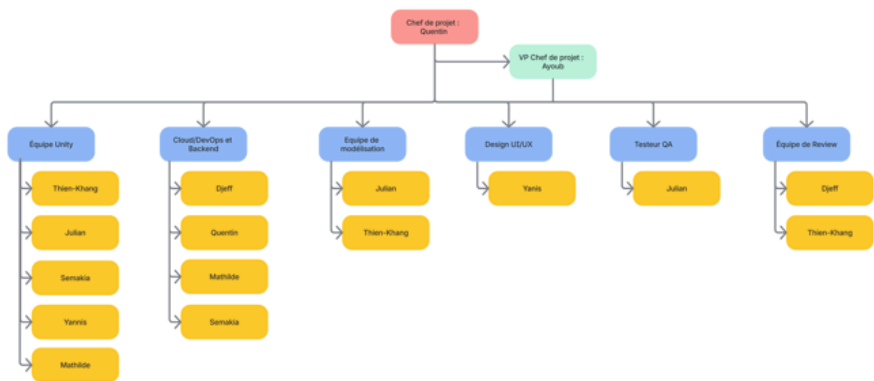
1. [Objectif](#)
2. [Rôles et responsabilités](#)
 - [2.1. Rôles spécifiques](#)
 - [2.2. Rôles récurrents](#)
3. [Méthodologie et outils](#)
 - [3.1. Méthodologie du projet](#)
 - [3.2. Outils utilisés](#)
4. [Normes de développement](#)
 - [4.1. Architecture et structure du code](#)
 - [4.2. Documentation du code](#)
 - [4.3. Normes Git](#)
 - [4.3.1. Hiérarchie des branches](#)
 - [4.3.2. Conventions de commit](#)
 - [4.3.3. Processus de merge](#)
5. [Processus de contrôle qualité](#)
 - [5.1. Normes et référentiels](#)
 - [5.2. Tests fonctionnels](#)
 - [5.2.1. Tests unitaires](#)
 - [5.2.2. Tests d'intégration](#)
 - [5.2.3. Tests de bout en bout \(E2E\)](#)
 - [5.3. Tests de sécurité](#)
 - [5.3.1. Tests de vulnérabilités](#)
 - [5.3.2. Tests d'authentification et autorisation](#)
 - [5.4 Tests de performances](#)
 - [5.5 Automatisation des tests](#)
6. [Gestion des anomalies et actions correctives](#)
 - [6.1 Stratégie de correction des anomalies](#)
7. [Amélioration continue et optimisation](#)
 - [7.1 Indicateurs de qualité \(KPI\)](#)
 - [7.2 Revues et amélioration](#)
8. [Documentation Onboarding](#)

1. Objectif

Ce document définit les dispositions mises en œuvre pour garantir la qualité tout au long du cycle de vie du projet. Il précise les méthodes, outils, responsabilités et contrôles permettant d’assurer la conformité aux exigences qualité définies.

2. Rôles et responsabilités

Afin d’assurer une bonne répartition de la charge de travail , chaque membre de l'équipe porte une double ou triple casquette en fonction des besoins ou son expertise . Chaque membre de l'équipe a donc un rôle récurrent qu’il exécute à principalement , et des rôles spécifiques .



2.1. Rôles spécifiques

Nom(s)	Rôle	Responsabilités
Quentin	Chef de projet	Pilotage global du projet, suivi du planning, coordination des équipes, gestion des risques, reporting.
Ayoub	Document / livrable	Supervision des livrables, processus d'onboarding

Yannis	Developpeur Unity	Intégration des assets, développement sous Unity
Sémakia	Developpeur Unity / Co-Responsable Backend	Développement client Unity, architecture et développement backend, API et services web, intégration client-serveur, tests unitaires et d'intégration
Djeff	Co- Responsable Backend / Co- Responsable Devops / Reviewer code / Document (Backend)	Architecture backend, infrastructure cloud (AWS/Azure), CI/CD et automatisation, monitoring et observabilité, review de code backend, documentation technique backend
Mathilde	Co-Responsable Devops Developpeur Unity	Infrastructure de déploiement, automatisation des tests, développement Unity, gestion des environnements,
Thien-Khang	Développeur Unity / Review de code / documentation	Développement gameplay Unity, review de code qualité, rédaction documentation développeur, respect des standards de codage,

Julian	Modélisateur Blender / Assets	
--------	--	--

2.2. Rôles récurrents

Nom(s)	Rôle	Responsabilités principales
Quentin	Chef de projet	Pilotage global du projet, suivi du planning, coordination des équipes, gestion des risques, reporting.
Ayoub	Concepteur / Dev Backend	Conception et modélisation UML, Développement des APIs REST/GraphQL, bases de données
Yannis	Design UX / UI	Conception des maquettes, validation des parcours utilisateurs, garant de l'expérience utilisateur.
Sémakia	Responsable Devops /	Automatisation des déploiements, mise en place des pipelines CI/CD, monitoring et alerting, gestion des environnements, sécurité infrastructure
Djeff	Responsable Cloud / Backend	Architecture et développement backend, gestion des services cloud, APIs et microservices, bases de données, sécurité

		backend, scalabilité et performances
Mathilde	Responsable Dev Backend /API	Développement des APIs REST/GraphQL, architecture des services backend, gestion des bases de données, tests d'intégration, documentation des APIs, optimisation des requêtes
Thien-Khang	Initiateur d'idées / Développeur Unity	Conception créative et innovation, développement Unity, prototypage rapide, veille technologique, proposition d'améliorations, développement de fonctionnalités expérimentales
Julian	Modélisateur Blender / Assets	Création et optimisation des modèles 3D, texturing et matériaux, préparation des assets pour Unity, respect des contraintes techniques, pipeline artistique

3. Méthodologie et outils

3.1 Méthodologie du projet

- **Organisation** : Sprints de deux semaines

- **Rituels :**

- Réunions hebdomadaires de type Project Review
- Compte rendu rédigé à chaque réunion
- Rétrospective en fin de sprint
- Démonstration en fin de sprint

Livrables attendus :

- Documentation technique (Confluence, README.md)
- Fonctionnalités livrées sur les branches develop
- Merge requests des fonctionnalités vers la branche main après validation

3.2 Outils utilisés

Type	Outils / Plateformes	Versions
Suivi de projet & documentation	Jira, Confluence	
Versionnage	GitLab	
Moteur de développement	Unity	6.0 2022 (Version la plus stable)
Modélisation 3D	Blender	4.4 (version la plus stable actuelle)
Qualité de code	SonarQube	Non définie
CI / Cloud	GitLab CI, Azure	
Communication interne	Comptes-rendus de réunion	

4. Normes de développement

4.1. Architecture et structure du code

Pour l'organisation de la structure de notre code, nous optons pour ***l'architecture MVC (Model-View-Controller)*** parce qu'elle est modulaire, maintenable et favorise la séparation des responsabilités. Cette architecture permet une meilleure organisation du code, facilite les tests unitaires et améliore la collaboration en équipe :

Structure des dossiers :

```
1 /Scripts/
2 |— Managers/           # Gestionnaires de système (GameManager, AudioManager, etc.)
3 |— Controllers/        # Contrôleurs de logique métier
4 |— Models/             # Modèles de données et entités
5 |— Views/              # Interfaces utilisateur et affichage
6 |— Utils/              # Utilitaires et helpers
7 |— Data/               # ScriptableObject et configurations
```

4.2. Documentation du code

Standard adopté : XML Documentation Comments (C# standard Microsoft)

Justification du choix :

- Support natif dans Visual Studio et Unity,
- Possibilité de générer automatiquement la documentation API avec des outils comme DocFX, Sandcastle ou XML Doc
- Format reconnu et largement utilisé dans l'écosystème .NET/Unity
- Documentation au plus près du code, mise à jour facilitée lors des modifications
- Warnings du compilateur en cas de documentation manquante ou incohérente

En-tête standard d'un fichier code (C#) :

```
1 /// Fichier : NomDuFichier.cs
2 /// Auteur : Ayoub Chouiref
3 /// Date : jj/mm/aaaa
4 /// Description : Brève description du contenu
5 /// Version : 1.0
6 /// Dernière modification : jj/mm/aaaa
```

Documentation obligatoire des fonctions (C#) :

```
1 /// <summary>
2 /// Description claire et concise de la fonction
3 /// </summary>
4 /// <param name="parametre1">Description du paramètre</param>
5 /// <param name="parametre2">Description du paramètre</param>
6 /// <returns>Description de la valeur de retour</returns>
7 /// <exception cref="ExceptionType">Conditions d'exception</exception>
8 public ReturnType NomDeLaFonction(Type parametre1, Type parametre2)
9 {
10     // Implémentation
11 }
```

Normes de nommage :

- **Classes** : PascalCase (ex: PlayerController)
- **Méthodes** : PascalCase (ex: GetPlayerData())
- **Variables** : camelCase (ex: playerHealth)
- **Constantes** : UPPER_SNAKE_CASE (ex: MAX_HEALTH)

- **Variables privées** : camelCase avec underscore (ex: `_isInitialized`)

4.3. Normes GIT

4.3.1. Hiérarchie des branches

Structure des branches :

- **main** : Branche de production stable
- **develop** : Branche de développement principal
- **feature/*** : Branches de fonctionnalités
- **fix/*** : Branches de correction de bugs
- **hotfix/*** : Branches de correctifs urgents

Format de nommage : `type_de_ticket/ID_du_ticket-Nom_du_ticket`

Types de tickets :

- **feat** : Développement d'une nouvelle fonctionnalité
- **fix** : Corrections de bugs
- **chore** : Maintenance, mise à jour des dépendances
- **refactor** : Refactorisation sans changement fonctionnel
- **hotfix** : Correctifs urgents en production
- **docs** : Mise à jour de la documentation
- **test** : Ajout ou modification de tests

Exemples :

- `feat/ESP-01-Create-Account`
- `fix/FB-23-Login-Error`
- `chore/FB-42-Update-Dependencies`

4.3.2. Conventions de commit

Format : `type(scope): description concise`

Composants :

- **Type** (obligatoire) : feat, fix, chore, refactor, docs, test, style, perf
- **Scope** (optionnel) : Partie concernée (auth, ui, api, db, player, etc.)
- **Description** (obligatoire) : Phrase courte à l'infinitif (max 50 caractères)

Exemples :

```
1 feat(auth): ajout de l'authentification OAuth
2 fix(ui): correction du bug d'affichage sur mobile
3 chore(deps): mise à jour des dépendances Unity
4 refactor(player): simplification du système de mouvement
```

4.3.3. Processus de merge

Pull Request (PR) / Merge Request (MR) :

- **Reviewers obligatoires** : Équipe de review
- **Types autorisés** : merge, squash, rebase (documenté au cas par cas)
- **Suppression des branches** après merge

Critères de validation :

- Tests unitaires passants
- Code coverage > 80%
- Respect des normes de codage
- Documentation à jour
- Pas de conflits de merge
- Pipeline CI/CD sans erreur

Template de Pull Request :

```
1 ## Description
2 Brève description des changements apportés
3
4 ## Type de changement
5 - [ ] Bug fix
6 - [ ] Nouvelle fonctionnalité
7 - [ ] Breaking change
8 - [ ] Documentation
9
10 ## Tests
11 - [ ] Tests unitaires ajoutés/mis à jour
12 - [ ] Tests d'intégration validés
13 - [ ] Tests manuels effectués
14
15 ## Checklist
16 - [ ] Code respecte les normes de l'équipe
17 - [ ] Documentation mise à jour
18 - [ ] Pas de code commenté inutile
19 - [ ] Variables et fonctions nommées clairement
```

5. Processus de contrôle qualité

5.1. Normes et référentiels

Notre processus de contrôle qualité s'appuie sur un ensemble de normes internationales reconnues :

- **ISO 9001:2015** - Système de management de la qualité

- **ISO/IEC 25010** - Modèle de qualité des systèmes et logiciels
- **ISO/IEC 29119** - Standard pour les tests logiciels
- **ISTQB** - Bonnes pratiques en tests logiciels

Cette approche multi-référentielle garantit l'efficacité de notre système qualité tout au long du cycle de vie du logiciel, en s'appuyant sur trois axes principaux : les tests fonctionnels, les tests de performances, les tests de sécurité .

5.2. Tests fonctionnels

5.2.1. Tests unitaires

- **Objectif** : Vérifier que chaque composant fonctionne correctement de manière isolée
- **Couverture cible** : Minimum 70% du code source (objectif 80%)
- **Framework** : Unity Test Framework (UTF) basé sur NUnit
- **Modes** : Edit Mode (scripts sans exécution) et Play Mode (pendant l'exécution)
- **Fréquence** : Exécution automatique à chaque commit (CI/CD)
- **Responsabilité** : Développeurs

5.2.2. Tests d'intégration

- **Tests par composants** : Interactions entre GameObjects et composants Unity
- **Tests système** : Validation avec APIs externes et bases de données
- **Environnement** : Unity Editor avec configuration de test dédiée

5.2.3. Tests de bout en bout (E2E)

- **Scénarios de gameplay** : Parcours complets de jeu et interactions utilisateur
- **Tests de régression** : Vérification de non-régression des nouvelles fonctionnalités
- **Tests d'interface** : Navigation dans les menus, HUD, interactions UI
- **Outils** : AltTester Unity SDK pour l'automatisation spécialisée Unity
- **Fréquence** : Manuellement selon les besoins

5.3. Tests de sécurité

5.3.1. Tests de vulnérabilités

- **Analyse statique** : Scan du code source
- **Analyse dynamique** : Tests en cours d'exécution
- **Tests de pénétration (optionnel)** : Simulation d'attaques réelles
- **Outils** : SonarQube

5.3.2. Tests d'authentification et autorisation

- **Gestion des sessions** : Validation des mécanismes de session
- **Contrôle d'accès** : Vérification des permissions et rôles utilisateur

5.4. Tests de performances

- **Tests de charge** : Évaluation de la capacité à supporter un grand nombre d'utilisateurs
- **Planification** : Tests effectués lorsque l'application sera terminée et en production
- **Outils** : Unity Profiler, Unity Performance Testing Package

5.5. Automatisation des tests

- **Analyse de code** : SonarQube pour l'analyse statique et la détection de bugs
- **CI/CD** : Pipelines automatisés GitLab CI déclenchés à chaque commit
- **Environnement** : Environnement dédié identique à la production

6. Gestion des anomalies et action correctives

Malgré un processus rigoureux, des défauts peuvent être détectés lors des tests. L'équipe utilise des outils comme Jira pour:

- **Répertorier et prioriser les anomalies** : en fonction de leur criticité (Bloquant, Majeur, Mineur, critique)
- **Mettre en place un processus de correction efficace**, incluant un suivi détaillé des correctifs.
- **Effectuer une analyse des causes profondes (Root Cause Analysis – RCA)** afin d'éviter la réapparition des mêmes erreurs dans les futures versions du logiciel.

6.1 Stratégie de Correction des Anomalies

Les stratégies de correction définies par l'équipe peuvent varier ou évoluer en fonction des besoins , de la charge de travail , et la criticité des anomalies détectées .

Anomalies Bloquantes ou Majeures :

- Si le délai de correction dépasse 24h
- Si l'anomalie bloque le développement en cours ou le fonctionnement de l'application
- Le chef de projet organise un point urgent avec le développeur et l'équipe de review (si disponible) pour décider de l'aide ou de la solution proposée

- Dans le cas aucune solution n'a été trouvé dans un délai raisonnable , prendre une décision radicale sur la continuité de cette fonctionnalité .

Anomalies Mineures :

- Prioriser d'abord les erreurs les plus urgentes
- Autoriser un certain niveau de dette technique dans un délai défini

7. Amélioration continue et optimisation

7.1. Indicateurs de qualité (KPI)

Indicateur	Objectif
Taux de couverture de test	≥ 80%
Nombre de bugs bloquants en prod	0
Délai de résolution des anomalies	>96h
Taux de réussite des builds CI	100%

7.2. Revues et amélioration

Type	Fréquence	Responsable	Objectif
Revue de code	À chaque merge	Équipe de review	Vérification des standards, détection des erreurs
Revue de ticket	Fin de ticket	Chef de projet	Validation des user stories

8. Documentation Onboarding

Pour simplifier l'arrivée des nouveaux collègues dans l'équipe et assurer leur intégration rapide, nous avons mis en place plusieurs outils de documentation et de support :

8.1. Documentation technique (Confluence / Git)

L'ensemble de la documentation technique est centralisé sur Confluence afin de garantir un accès clair, structuré et collaboratif à tous les membres de l'équipe.

Elle comprend notamment :

- **Guide d'architecture** : schémas UML, description des composants Unity, structure de la base de données, logique multijoueur, etc.
- [Règles de nommage et conventions de code](#) : standards de développement appliqués au projet pour garantir lisibilité et maintenabilité.
- [Workflow Git](#) : stratégie de branches (`develop` , `main` , `feat/` , `fix/` ...), gestion des merge requests, et bonnes pratiques de versioning.
- **Stratégie de tests** : organisation des tests unitaires, fonctionnels, E2E, spécifiques à la 3D ou à la réalité augmentée (AR).
- **CI/CD et déploiement** : procédures de pipeline (GitLab CI), intégration continue, déploiement sur Azure, et automatisations associées.

8.2. Tutoriels d'arrivée pratiques

Des **mini-guides pratiques** sont disponibles pour faciliter l'intégration des nouveaux membres. Ces tutoriels sont évolutifs et adaptés aux outils que nous utilisons.

Ils couvrent notamment :

- **Utilisation de Git & GitLab** : cloner un dépôt, créer une branche, faire une merge request, etc.
- **Lancement des tests en local** : exécuter les tests unitaires et fonctionnels sur l'environnement local.
- **Démarrage d'une scène Unity** : ouvrir le projet, lancer une scène de test, vérifier les composants.
- **Intégration d'un ticket Jira dans le flux de développement** : gestion des tâches, lien entre ticket et code, checklist.

- **Accès aux canaux de communication** : rejoindre les espaces Discord, Teams ou autres outils utilisés.

3. Organisation et accompagnement

L'intégration est soutenue par un cadre humain et organisationnel solide :

- Une réunion d'accueil avec le chef de projet et les référents techniques
- La nomination d'un binôme d'intégration disponible pour répondre aux premières questions
- Des tickets Jira détaillés, avec sous-tâches et checklists pour guider les premières missions
- Une revue systématique des premières merge requests avec des retours constructifs pour favoriser l'apprentissage

9. Références

- [GitHub Flow](#)
- [Conventional Commits](#)
- [DevOps by Microsoft](#)
- [Google Code Review Practices](#)